

COP4533

ALGORITHM ABSTRACTION & DESIGN PROGRAMMING PROJECT

1 Problem Definition

Consider an inspection robot crossing a row of n service panels on an orbital solar array. The robot begins at a docking position numbered 0 and must reach panel n . From any position, the robot may jump forward by at least one and at most k positions. Landing on panel i , where $1 \leq i \leq n$, consumes c_i units of energy. The robot does not pay a cost at position 0, cannot move backward, and must land on panel n . The total cost of a route is the sum of the costs of all panels on which the robot lands. Your goal is to determine a valid route having minimum total cost.

EXAMPLE 1: $n = 7$, $k = 3$,

$c = [7, 2, 9, 4, 1, 8, 3]$

SOLUTION: Land on $[2, 5, 7]$;

Energy = $2 + 1 + 3 = 6$

EXAMPLE 2: $n = 8$, $k = 3$,

$c = [20, 18, 16, 14, 12, 10, 8, 6]$

SOLUTION: Land on $[3, 6, 8]$;

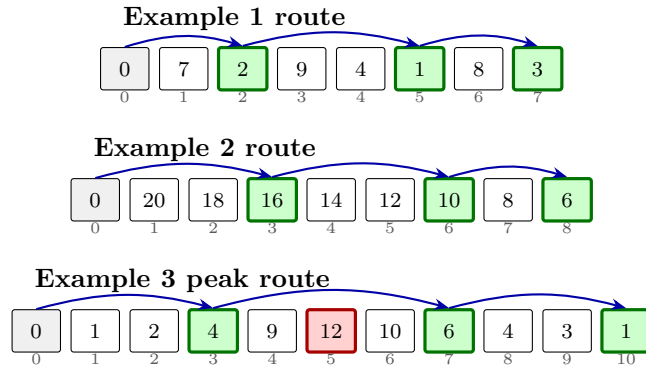
Energy = $16 + 10 + 6 = 32$

EXAMPLE 3: $n = 10$, $k = 4$,

$c = [1, 2, 4, 9, 12, 10, 6, 4, 3, 1]$

SOLUTION: Land on $[3, 7, 10]$;

Energy = $4 + 6 + 1 = 11$



Numbers inside boxes are landing costs; numbers below are panel indices. Green boxes form the route; the red box is the peak panel.

Below is the formal problem definition, starting with the general version, followed by two special cases. The special cases include input restrictions, which may allow for more efficient solutions.

PROBLEMG Given positive landing costs $c_1, \dots, c_n$ and a positive integer k , find a route $0 = i_0 < i_1 < \dots < i_m = n$ such that $1 \leq i_r - i_{r-1} \leq k$ for every r , minimizing $\sum_{r=1}^m c_{i_r}$.

PROBLEMS1 Solve PROBLEMG when the landing costs are monotonically non-increasing:

$$c_i \geq c_j \quad \forall i < j.$$

(The costs in Example 2 satisfy this restriction.)

PROBLEMS2 Solve PROBLEMG when there is a unique peak index p , where $1 < p < n$, such that $c_p > c_i$ for every $i \neq p$, and

$$c_1 \leq c_2 \leq \dots \leq c_p \quad \text{and} \quad c_p \geq c_{p+1} \geq \dots \geq c_n.$$

(The costs in Example 3 satisfy these restrictions.)

2 MILESTONE 1 - Greedy Algorithms

For the first part of the project, you will design, analyze and implement greedy algorithms to solve the special cases of the problem. You will conduct an experimental study on the

performance of your algorithms. You are also required to construct examples showing how an algorithm developed for a special case of the problem might not work for the general version of the problem.

2.1 Design, Analysis, and Implementation Tasks

ALGORITHM1 Design a $\Theta(n)$ time greedy algorithm for solving PROBLEMS1.

ANALYSIS1 Prove the correctness of your greedy ALGORITHM1 for solving PROBLEMS1.

QUESTION1 Give an input example showing that ALGORITHM1 does not always solve PROBLEM G.

QUESTION2 Give an input example showing that ALGORITHM1 does not always solve PROBLEMS2.

PROGRAM1 Give an implementation of your greedy ALGORITHM1.

ALGORITHM2 Design a $\Theta(n)$ time greedy algorithm for solving PROBLEMS2.

ANALYSIS2 Prove the correctness of your greedy ALGORITHM2 for solving PROBLEMS2.

PROGRAM2 Give an implementation of your greedy ALGORITHM2.

2.2 Experimental Comparative Study

Test your implementations extensively for correctness and performance. For this purpose, you should create randomly generated input files of various sizes appropriate to each program. Then, conduct and present the following performance comparisons. For each comparison, generate a two-dimensional plot of running time (y-axis) against input size (x-axis). Include the plots in your report along with your comments and observations. *Note: The size of your data sets should be such that the plots clearly illustrate how the running time of your algorithms grows with the input size*

PLOT1 Plot the running time of PROGRAM1 with respect to varying input size.

PLOT2 Plot the running time of PROGRAM2 with respect to varying input size.

2.3 Report

Prepare a report that describes the design and analysis of your algorithms, presents the results of the experimental study, and summarizes your learning experience. Your report should include, but is not limited to, the following sections.

- **Team Members.** You may choose to work alone or in a team of up to three students on this assignment. If you choose to work in a team, clearly state the name and contribution of each team member.
- **Algorithm Design and Analysis.** Give a clear description of your algorithm designs and their analyses. You may also include pseudocode. Make sure to provide the answers to QUESTION1 and QUESTION2.
- **Experimental Study.** Present the results of your experimental study. Include all plots along with any explanations and comments you wish to provide.
- **Conclusion.** Summarize your learning experience on the first component of the project. For each programming task, comment on the ease of implementation and other potential technical challenges.

2.4 Deliverables

The following contents are required for the MILESTONE1 submission:

- **Implementation:** Submit the implementation code on Gradescope. Refer to Section 5 for more details. Include detailed comments next to each non-trivial block of code.
- **Report:** Submit the report in PDF format on Gradescope.

3 MILESTONE 2 - Dynamic Programming Algorithms

For the second part of the project, you will design, analyze and implement dynamic programming algorithms to solve the general case of the problem. You will conduct an experimental study on the performance of your algorithms.

3.1 Design, Analysis, and Implementation Tasks

ALGORITHM3 Design a $O(2^n)$ time naive recursive algorithm for solving PROBLEM G.

ANALYSIS3 Give an analysis (correctness and running time) of ALGORITHM3 for solving PROBLEM G.

ALGORITHM4 Design a $\Theta(nk)$ or $\Theta(k + (n - k)k)$ time dynamic programming algorithm for solving PROBLEM G.

ANALYSIS4 Give an analysis (correctness and running time) of ALGORITHM4 for solving PROBLEM G.

ALGORITHM5 Design a $\Theta(n)$ time dynamic programming algorithm for solving PROBLEM G.

ANALYSIS5 Give an analysis (correctness and running time) of ALGORITHM5 for solving PROBLEM G.

PROGRAM3 Implementation of ALGORITHM3.

PROGRAM4A Top-down recursive implementation of ALGORITHM4 using memoization.

PROGRAM4B Iterative bottom-up implementation of ALGORITHM4.

PROGRAM5 Implementation of ALGORITHM5.

3.2 Experimental Comparative Study

Test your implementations extensively for correctness and performance. For this purpose, you should create randomly generated input files of various sizes appropriate to each program. Then, conduct and present the following performance comparisons. For each comparison, generate a two-dimensional plot of running time (y-axis) against input size (x-axis). You need to find a suitable way to create the plot with both n and k . For example, you could set $k = \lceil n/10 \rceil$. Include the plots in your report along with your comments and observations. *Note: The size of your data sets should be such that the plots clearly illustrate how the running time of your algorithms grows with the input size*

PLOT3 Plot the running time of PROGRAM3 with respect to varying input size.

PLOT4 Plot the running time of PROGRAM4A with respect to varying input size.

PLOT5 Plot the running time of PROGRAM4B with respect to varying input size.

PLOT6 Plot the running time of PROGRAM5 with respect to varying input size.

PLOT7 Overlay PLOTS 3, 4, 5, 6 and contrast the performance of PROGRAMS 3, 4A, 4B, 5.

PLOT8 Overlay PLOTS 4, 5 and contrast the performance of PROGRAMS 4A and 4B.

In addition to testing the running time performance of the algorithms described above, you are asked to conduct experiments to test the quality of the output of greedy ALGORITHM1, when it is run on the general case of the problem PROBLEM_G. Note that such solution is not guaranteed to be optimal. Your task is to determine how different it is from the optimal. Make sure to conduct enough experiments with many random data sets. For x -axis, simply use n as the varying input size. Let E_g be the energy cost produced by ALGORITHM1, and let E_o be the optimal energy cost determined by any of PROGRAMS 3, 4A, 4B, 5.

PLOT9 Plot the relative excess cost

$$\frac{E_g - E_o}{E_o}$$

against input size n .

3.3 Report

Prepare a report that describes the design and analysis of your algorithms, presents the results of the experimental study, and summarizes your learning experience. Your report should include, but is not limited to, the following sections.

- **Team Members.** You may choose to work alone or in a team of up to three students on this assignment. If you choose to work in a team, clearly state the name and contribution of each team member.
- **Algorithm Design and Analysis.** Give a clear description of your algorithm designs and their analyses. Include the recursive formulation expressing optimal substructure for the dynamic programming algorithms. You may also include pseudocode.
- **Experimental Study.** Present the results of your experimental study. Include all plots along with any explanations and comments you wish to provide.
- **Conclusion.** Summarize your learning experience on the second component of the project. For each programming task, comment on the ease of implementation and other potential technical challenges.

3.4 Deliverables

The following contents are required for the MILESTONE2 submission:

- **Implementation:** Submit the implementation code on Gradescope. Refer to Section 5 for more details. Include detailed comments next to each non-trivial block of code.
- **Report:** Submit the report in PDF format on Gradescope along with the source files.

4 MILESTONE 3 - Video Presentation

Prepare a video of length between 5 and 7 minutes that presents all your results (design, analysis, experiments, and learning experience). We will provide more details regarding the submission and grading criteria for this component of the project.

5 Language/Input/Output Specifications

You may use C++, Java, or Python. Starter code for each language is provided (they differ only in filenames and class names). Add your implementation to the provided starter code in the language of your choice. Do **not** modify the filenames or class names in the given code. You must upload a file for each program on Gradescope (1 file per program). Do **not** include any extra files. You will receive instant feedback for a few example test cases to verify that your code follows the correct specifications.

The autograder on Gradescope runs on an Ubuntu 22.04 image. C++ code will be compiled using g++ 11.4.0 with the ‘-std=gnu++17’ compile flag. Java code will be compiled and evaluated using OpenJDK 17. Python code will be evaluated using Python 3.10.

For convenience, you can assume that $1 \leq k \leq n < 10^5$, and $\forall i \ 1 \leq c_i < 10^5$. Test cases will be generated to accommodate the required time complexity. The generated test cases will also have a unique solution.

Input. Your program will read input from standard input (`stdin`) in the following order:

- Line 1 consists of two integers n and k separated by a single space.
- Line 2 consists of n integers $c_1, \dots, c_n$ separated by a single space(not including 0).

Output. Your program should print to standard output (`stdout`) in the following order:

- Line 1 contains the minimum total energy cost.
- Line 2 contains the number of panels m visited(including last)
- The next m lines contain the indices of the picked panels $i_1, \dots, i_m$ in increasing order(not including 0), where m is the number of panels chosen.

6 AI Use Disclosure

AI is permitted exclusively for quality feedback and grammar or writing structure. Each report must contain a clearly labeled AI Use Disclosure. If generative AI tools were used, identify the tools, how they were used, and how the resulting material was checked; otherwise, state that no generative AI tools were used. Disclosure does not authorize otherwise prohibited assistance.

7 Grading Policy

Milestones 1, 2, and 3 will weigh **30%**, **50%**, and **20%** of your total project grade, respectively. Outstanding submissions may receive up to **20%** bonus points.

For Milestones 1 and 2, grades will be based on the correctness and efficiency of the algorithms and the quality of the experimental study and report:

- **Program 60%.** Correct and efficient design, implementation, and execution. Code should be well structured, readable, and appropriately commented.
- **Report 40%.** Quality, clarity, organization, and presentation of the write-up on algorithm design, analysis, programming experience, and experimental study.

Bonus points primarily recognize exceptional presentation quality, concise analysis, highly readable code, and deeper experimental insights or visualizations beyond the baseline requirements.